컴퓨터프로그래밍 2

Odd and Even Game

홀짝 게임

영원 조

20231508 강유영

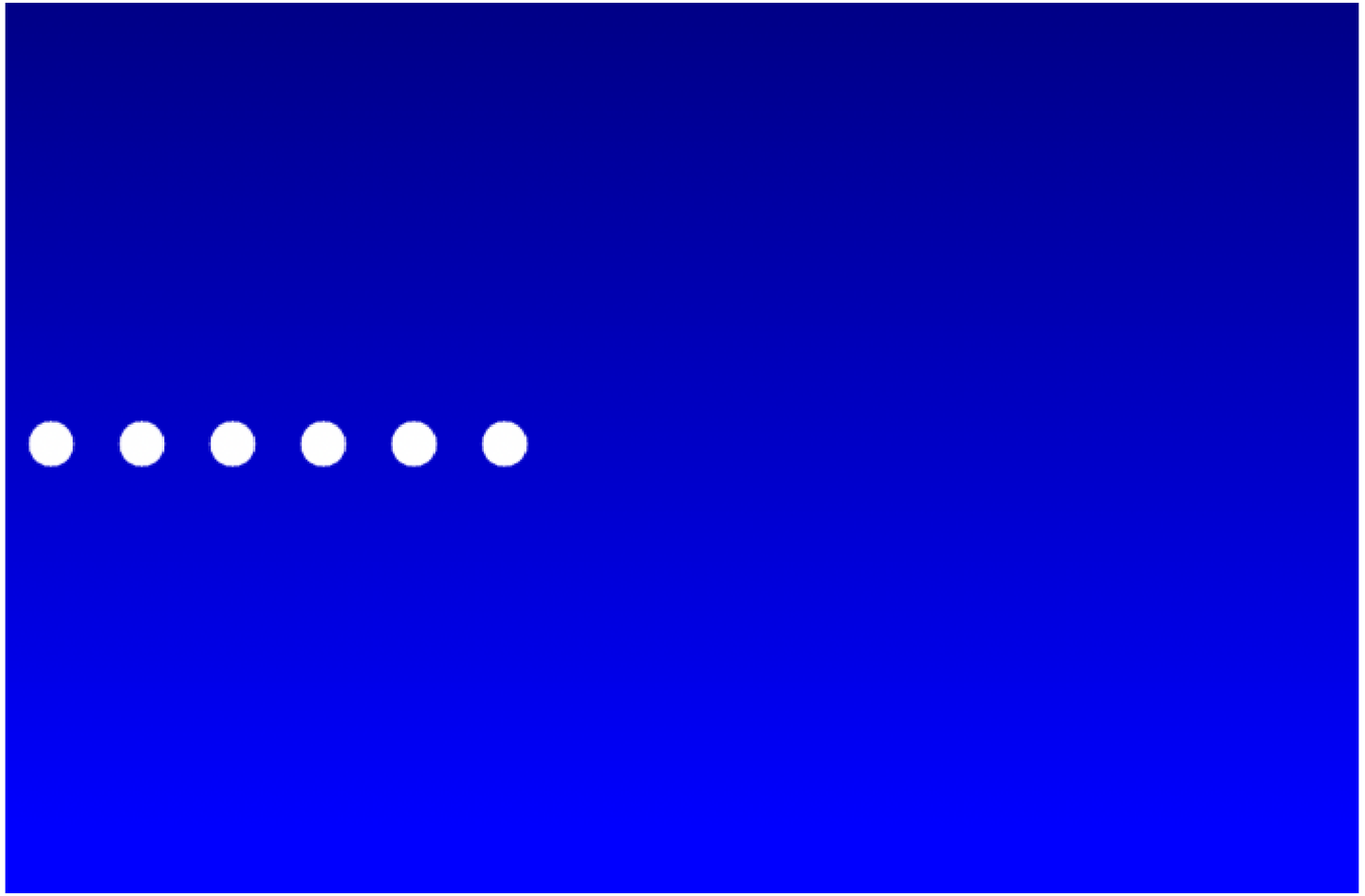
20231528 김소원

01

게임 규칙

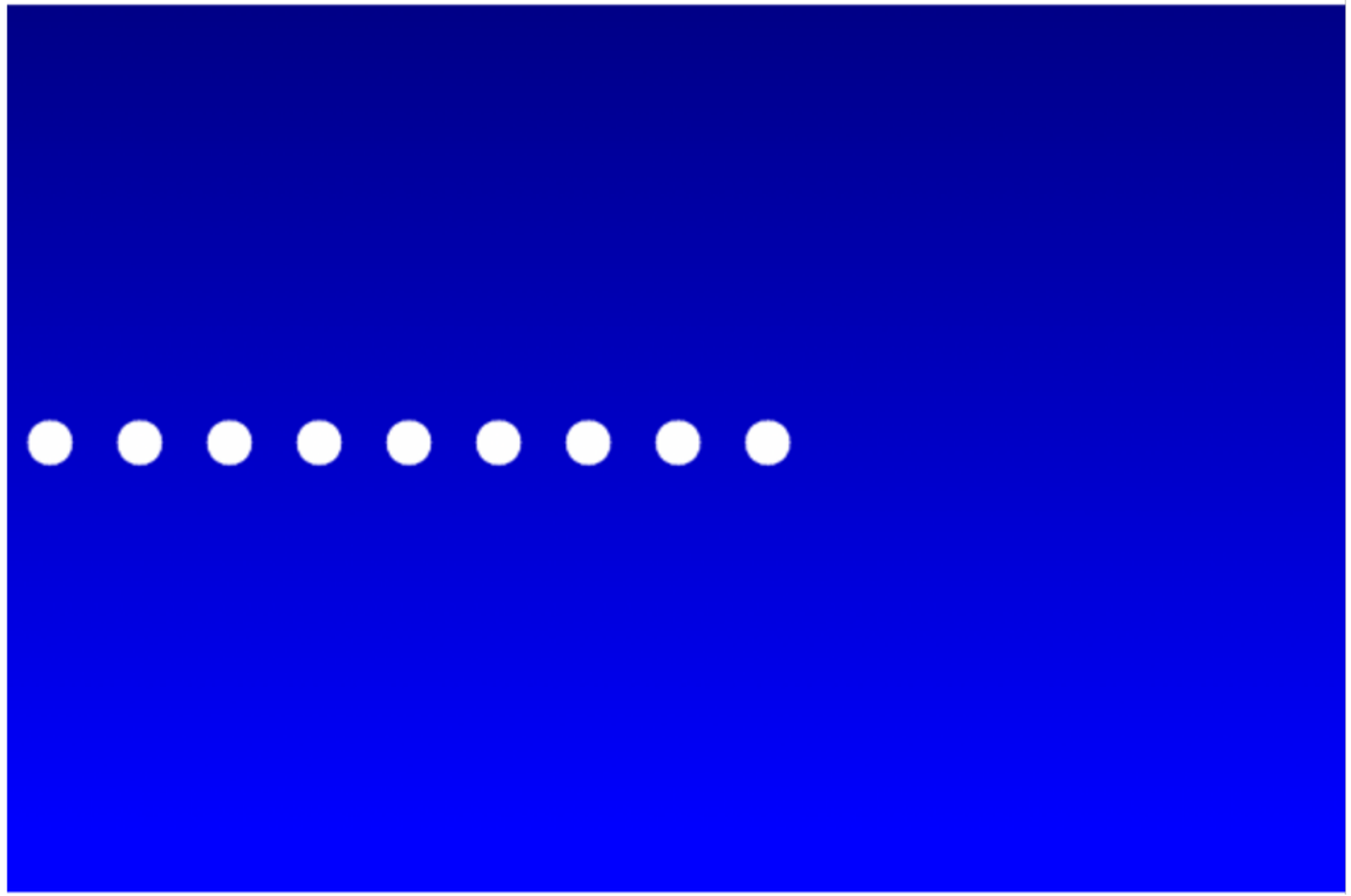
- 플레이어와 컴퓨터는 각각 6개의 구슬을 가지고 시작
- 컴퓨터와 플레이어가 번갈아가며 자신이 현재 가지고 있는 구슬 수 내에서 구슬을 걸고, 건 구슬의 수가 짝수인지 홀수인지 맞힘
- 짝수, 홀수를 맞히면 걸려있는 구슬을 가져감
- 짝수, 홀수를 틀리면 걸려있는 구슬 만큼 빼앗김
- 한쪽이 구슬을 모두 얻으면 게임이 끝남

01



```
홀수입니까? 짝수입니까? (홀수면 1, 짝수면 2를 입력하시오):
```

01



```
홀수입니까? 짝수입니까? (홀수면 1, 짝수면 2를 입력하시오): 1
이겼습니다!(현재 구슬 수: 9)
```

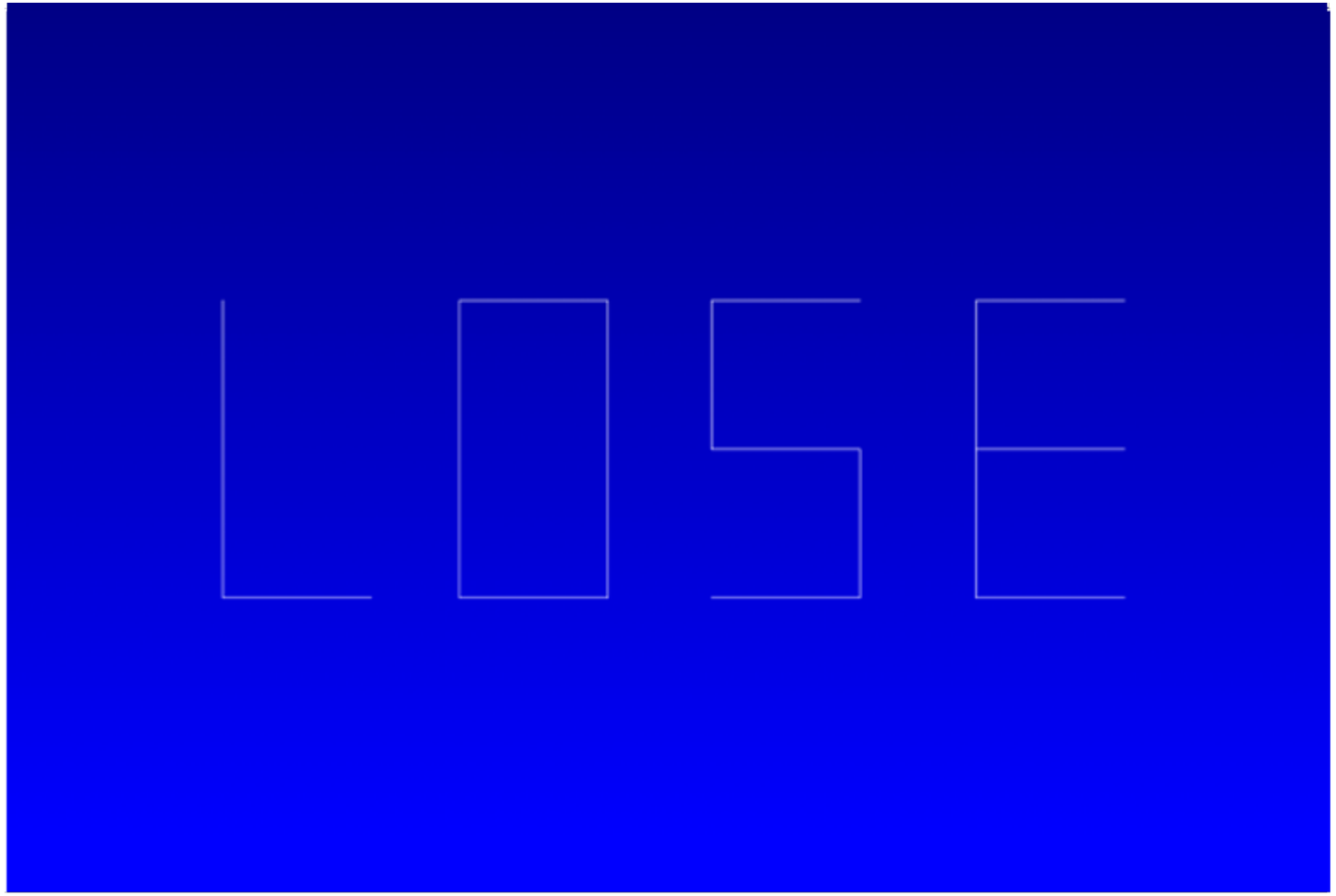
01

```
구슬 몇 개를 걸겠습니까? (현재 구슬 수: 9): 4
졌습니다!(현재 구슬 수: 5)
```

01

당신은 모든 구슬을 얻었습니다!

01



당신은 모든 구슬을 잃었습니다!

02

코드

게임 알고리즘 (main)

구슬, WIN, LOSE 출력 함수

화면 갱신 함수

02

게임 알고리즘 코드

```
while (playerbeads > 0 && playerbeads < 12) {...}
```

플레이어의 구슬 수가 1~11개 일 때 게임 알고리즘 반복

```
int computernumber = rand() % computerbeads + 1;
bool is_c_odd = (computernumber % 2 == 1);
```

```
int computernumber_2 = rand() % 2;   //컴퓨터가 낸 랜덤값으로 0 또는 1를 출력한다.
```

랜덤함수로 컴퓨터가 거는 구슬 수/짝수, 홀수 선택 여부를 얻는다

03

화면 출력 함수

구슬 출력

```
void drawCircle(png_bytep *row_pointers, int x, int y, int radius) {
for (int dy = -radius; dy <= radius; dy++) {
for (int dx = -radius; dx <= radius; dx++) {
if (x + dx >= 0 && x + dx < imageWidth && y + dy >= 0 && y + dy < imageHeight) {
if (dx * dx + dy * dy <= radius * radius) {
row_pointers[y + dy][4 * (x + dx)] = 255;
row_pointers[y + dy][4 * (x + dx) + 1] = 255;
row_pointers[y + dy][4 * (x + dx) + 2] = 255;
row_pointers[y + dy][4 * (x + dx) + 3] = 255;
```

```
void drawBeads(png_bytep *row_pointers, int numBeads) {
const int circleRadius = 20;
const int padding = 40;
for (int i = 0; i < numBeads; i++) {
int centerX = padding + i * (2 * circleRadius + padding);
int centerY = imageHeight / 2;
drawCircle(row_pointers, centerX, centerY, circleRadius);
 }
}
```

drawCircle()

drawBeads()

03

화면 출력 함수

WIN, LOSE 출력을 위해 drawLine()를 만들었다

```
void drawLine(png_bytep *row_pointers, int x1, int y1, int x2, int y2) {
    int dx = abs(x2 - x1);
    int dy = abs(y2 - y1);
    int sx, sy;

    if (x1 < x2) {
        sx = 1;
    } else {
        sx = -1;
    }

    if (y1 < y2) {
        sy = 1;
    } else {
        sy = -1;
    }

    int err = dx - dy;
```

```
    while (true) {
        if (x1 >= 0 && x1 < imageWidth && y1 >= 0 && y1 < imageHeight) {
            row_pointers[y1][4 * x1] = 255;
            row_pointers[y1][4 * x1 + 1] = 255;
            row_pointers[y1][4 * x1 + 2] = 255;
            row_pointers[y1][4 * x1 + 3] = 255;
        }

        if (x1 == x2 && y1 == y2) {
            break;
        }

        int e2 = 2 * err;

        if (e2 > -dy) {
            err = err - dy;
            x1 = x1 + sx;
        }

        if (e2 < dx) {
            err = err + dx;
            y1 = y1 + sy;
        }
    }
}
```

04

화면 갱신 함수

입력마다 다른 output.png 생기면 메모리 낭비

->기존 파일 위에 새로운 이미지 덧씌우는 함수 사용

updateImage(), saveImage()